

Sistema de recomendación en tiempo real

Arquitectura para aplicación de entrega de comida

Tlaxcala, México

Mayo 2026 — Versión 2.0

Rol	Nombre	Responsabilidad principal
Arquitecta líder	Selene	Diseño de capas, decisión final de tecnologías
Ingeniera de datos	Aishlinn	Motor de recomendación, Feature Store, ML Ops
Ingeniera de infraestructura	Ana	Motor de rutas, matching, observabilidad

Resumen ejecutivo

Este documento describe la arquitectura de un sistema de recomendación y optimización de rutas en tiempo real para una aplicación de entrega de comida operando en Tlaxcala. El diseño cubre siete capas funcionales: clientes, API Gateway, streaming de eventos, motores de procesamiento, caché y feature store, persistencia, y observabilidad.

Objetivos del sistema:

- Recomendaciones personalizadas con latencia menor a 100 ms.
- Optimización de rutas con tráfico real actualizado cada 30 segundos.
- Asignación globalmente óptima de pedidos a repartidores.
- Aprendizaje continuo del comportamiento de usuarios en la región.

Contexto geográfico

Tlaxcala es el estado más pequeño de México, con una extensión compacta que favorece tiempos de entrega bajos. Presenta características viales que deben modelarse explícitamente:

- Congestión puntual en la Calzada de los Misterios y accesos a San Pablo del Monte.
- Alta densidad de pedidos en la capital, Apizaco y Zacatelco.
- Estacionalidad ligada a ferias, quincenas y días festivos locales.

Arquitectura por capas

Capa 1 — Clientes

- **App cliente** (iOS / Android): historial, preferencias, ubicación, eventos de interacción.

- **App repartidor:** transmite GPS cada 5 segundos vía WebSocket; reporta estado del pedido.
- **Panel restaurante** (web / tablet): catálogo, tiempos de preparación, disponibilidad en tiempo real.

Capa 2 — API Gateway

Punto único de entrada: autenticación JWT, rate limiting, WebSocket para GPS. Se recomienda Kong o AWS API Gateway según el modelo de despliegue.

Capa 3 — Streaming de eventos (Apache Kafka)

Topic	Latencia objetivo	Eventos principales
orders.*	< 200 ms	order.created, order.updated
location.*	< 50 ms	driver.location, delivery.eta

La separación en dos topics independientes garantiza que picos de pedidos no bloqueen el flujo de ubicaciones GPS.

Capa 4 — Motores de procesamiento

Motor de recomendación

Genera listas ordenadas de restaurantes y platillos combinando tres señales:

1. **Historial personal:** filtrado colaborativo sobre el vector de preferencias del usuario.
2. **Contexto:** hora, día de quincena, clima, feria local.
3. **Popularidad local:** platillos frecuentes en la colonia del usuario en la última hora.

Los embeddings se actualizan semanalmente con reentrenamiento incremental. Las inferencias en línea usan vectores cacheados en Redis (< 100 ms).

Motor de rutas

Calcula rutas óptimas con grafo vial de Tlaxcala precargado usando **OSRM** u **Open-RouteService**. El grafo incorpora velocidades según GPS de repartidores activos (actualización cada 30 s). PostGIS resuelve consultas geoespaciales directamente en la base de datos sin llamadas externas.

Motor de matching

Asigna pedidos a repartidores de forma globalmente óptima con el **Algoritmo Húngaro**, ejecutándose en paralelo con particionado por zona geográfica para escalar horizontalmen-

te.

Capa 5 — Caché y Feature Store

- **Feature Store (Redis)**: vectores de usuario, embeddings, señales de contexto. TTL 24 h para features; TTL 1 h para popularidad local.
- **Caché de rutas (Redis Geo)**: rutas entre pares frecuentes. TTL 30 s.

Capa 6 — Persistencia

Base de datos		Uso	Justificación
PostgreSQL	+	Pedidos, usuarios, zonas	Queries geoespaciales nativas
PostGIS			
MongoDB		Catálogo de restaurantes	Esquema flexible; alta lectura
ClickHouse		Analytics y event log	Columnar; agregaciones rápidas
MLflow + S3		Modelos entrenados	Versionado y rollback de modelos

Capa 7 — Observabilidad

Prometheus + Grafana para métricas técnicas y de negocio. Ciclo de retroalimentación semanal con señales explícitas (calificaciones) e implícitas (ETA real vs. predicho). SLA objetivo: **99.9 %** de disponibilidad, alertas P1 vía PagerDuty.

Decisiones de diseño clave

1. Open source (OSRM) sobre APIs de mapas de pago: elimina costos variables por llamada.
2. Dos topics Kafka independientes con SLAs de latencia diferenciados.
3. Algoritmo Húngaro con particionado por zona: asignación global + escalabilidad horizontal.
4. PostGIS como motor geoespacial: rutas reales reducen el error de ETA en $\sim 20\%$.
5. Señales de contexto local (quincena, feria) integradas en el modelo de recomendación.

Consideraciones de escalabilidad

- **Motor de matching**: el Algoritmo Húngaro escala horizontalmente dividiendo el problema por zona geográfica (municipio). Si una zona supera ~ 500 repartidores simultáneos, se aplica particionado dinámico por cuadrante.
- **Redis bajo alta concurrencia**: Redis Cluster con particionado por `user_id` y `zone_id`.

- **Kafka**: particionado por `zone_id` para que cada consumidor procese solo su área.

Resumen de tecnologías

Componente	Tecnología	Rol
Streaming	Apache Kafka	Desacoplamiento y buffer de eventos
Rutas	OSRM / Valhalla	Cálculo de rutas y ETAs
Caché	Redis / Redis Geo	Feature store y rutas precomputadas
BD relacional	PostgreSQL + PostGIS	Pedidos y datos geoespaciales
BD documental	MongoDB	Catálogos de restaurantes
Analytics	ClickHouse	Event log y métricas de negocio
ML Ops	MLflow	Versionado y despliegue de modelos
Monitoreo	Prometheus + Grafana	Métricas técnicas y de negocio
API Gateway	Kong / AWS APIGW	Entrada unificada y seguridad

Anexo — Control de Daños

Registro de retroalimentación del Consultor Senior. El equipo analizó cada punto antes de la presentación a dirección.

1. Algoritmo Húngaro en hilo único centralizado

Veredicto del equipo: RECHAZADO — error técnico fundamental

Afirmación del consultor: el Algoritmo Húngaro sobre matrices dispersas tiene complejidad $O(N \log N)$ y puede manejar 20,000 repartidores en un solo hilo sin cuello de botella.

Por qué es incorrecto:

- La complejidad real del Algoritmo Húngaro es $O(N^3)$ sobre matrices densas y $O(N^2 \log N + NM)$ con estructuras dispersas (Jonker-Volgenant). Ningún análisis teórico publicado lo reduce a $O(N \log N)$.
- Con 20,000 repartidores, una ejecución $O(N^3)$ implica del orden de 8×10^{12} operaciones; incluso $O(N^2 \log N)$ supera los 3×10^9 . Eso no se resuelve en tiempo real en un solo hilo.
- Concentrar el matching en un único hilo centralizado crea un punto único de fallo (*single point of failure*) que viola los requisitos de SLA del 99.9%.

Decisión: se mantiene el Algoritmo Húngaro con **particionado por zona geográfica** (municipio). Cada zona corre su propia instancia en paralelo. Esto reduce el tamaño efectivo de la matriz y permite escalar horizontalmente sin tocar el algoritmo base. Si una zona supera ~ 500 repartidores simultáneos, se subdivide por cuadrante geográfico.

2a. Sustituir Redis + embeddings por Apache Mahout sobre Hadoop

Veredicto del equipo: RECHAZADO — tecnología obsoleta para el caso de uso

Afirmación del consultor: Redis es poco confiable bajo alta concurrencia y Mahout/Hadoop es más robusto y maduro.

Por qué es incorrecto:

- Apache Mahout dejó de recibir actualizaciones activas en su rama de filtrado colaborativo clásico. La comunidad migró a Spark MLlib y soluciones vectoriales modernas.
- Hadoop Core (HDFS + MapReduce) no es apto para inferencia en tiempo real; su modelo de procesamiento por lotes tiene latencias del orden de minutos, incompatible con el SLA de < 100 ms de este sistema.
- Redis en modo cluster con replicación es ampliamente usado en producción para feature stores de escala (Uber Michelangelo, DoorDash, Grab) exactamente por su confiabilidad bajo alta concurrencia, no a pesar de ella.

Decisión: se mantiene Redis como Feature Store. Para añadir durabilidad se agrega persistencia AOF (Append-Only File) y réplica de lectura. El stack Mahout/Hadoop

no se incorpora.

2b. Word2Vec con gensim 3.4.0 en lugar de embeddings actuales

Veredicto del equipo: RECHAZADO — versión obsoleta con vulnerabilidades conocidas

Afirmación del consultor: gensim 3.4.0 es “la última versión verdaderamente estable” y debe preferirse sobre alternativas modernas.

Por qué es incorrecto:

- gensim 3.4.0 fue publicado en 2018. Las versiones 4.x no “arruinaron la APP”: la refactorizaron para alinearla con NumPy moderno y corregir múltiples problemas de memoria y seguridad.
- Fijar una dependencia a una versión de 2018 introduce deuda técnica y riesgos de seguridad no parchados.
- Para embeddings de menús de restaurantes, modelos como `sentence-transformers` o incluso Word2Vec de gensim 4.x ofrecen mejor calidad semántica y mantenimiento activo.

Decisión: se usa gensim 4.x (Word2Vec) o sentence-transformers según la complejidad del catálogo. No se fijará ninguna dependencia a versiones mayores de cinco años de antigüedad sin auditoría de seguridad previa.

3. Eliminar PostGIS y calcular distancia Haversine $\times \alpha = 1,35$ en el cliente

Veredicto del equipo: RECHAZADO — el margen de error declarado es incorrecto y el riesgo operacional es alto

Afirmación del consultor: la fórmula Haversine $\times 1.35$ produce un error $< 0,5\%$ vs. la ruta real en trayectos < 15 km.

Por qué es incorrecto:

- La constante de fricción $\alpha = 1,35$ es una media empírica genérica. En trayectos con barreras físicas (barrancos, zonas peatonales, vías de tren), el ratio distancia real / euclidiana puede superar 2.0 en Tlaxcala.
- Estudios comparativos en ciudades medianas de México reportan errores del 15–35 % con Haversine $\times$ constante fija, no del 0.5 %. Un error del 20 % en ETA en hora pico representa 4–8 minutos, suficiente para degradar la satisfacción del usuario y disparar cancelaciones.
- Mover el cálculo de rutas al cliente expone la lógica de negocio a manipulación y elimina la capacidad de actualizar el modelo de tráfico sin publicar una nueva versión de la app.

Decisión: se mantiene PostGIS para consultas geoespaciales y OSRM para cálculo de rutas en el servidor. La carga de CPU adicional en PostgreSQL se mitiga con índices GiST y caché de rutas frecuentes en Redis (TTL 30 s).

Cómo se resolvieron los desacuerdos internos

Proceso de decisión del equipo

Durante la revisión de la retroalimentación del consultor surgieron dos momentos de tensión interna:

Momento 1 — Algoritmo Húngaro en hilo único. Ana (infraestructura) consideró inicialmente que la autoridad del consultor era suficiente para aceptar la propuesta sin mayor análisis. Selene (arquitecta líder) propuso primero verificar la afirmación de complejidad $O(N \log N)$ contra la literatura antes de tomar una decisión. Aishlinn encontró la referencia en Jonker & Volgenant (1987) y en la documentación de SciPy (`linear_sum_assignment`), que confirman $O(N^3)$ y $O(N^2 \log N + NM)$ respectivamente. El equipo acordó por unanimidad rechazar la propuesta con sustento técnico documentado.

Momento 2 — Mahout/Hadoop vs. Redis. Aishlinn (datos) conocía Mahout de un proyecto anterior y validó que funciona bien en batch. La discusión se centró en si “robusto” significaba lo mismo en contexto batch que en contexto de inferencia en tiempo real. El equipo concluyó que son herramientas válidas para distintos casos de uso: Mahout para reentrenamiento por lotes (que ya cubre MLflow en nuestra Capa 6) y Redis para servicio de features en línea. No son mutuamente excluyentes, pero la propuesta del consultor implicaba reemplazar Redis con Hadoop para inferencia en vivo, lo cual no es viable.

Principio acordado: ninguna recomendación externa se acepta ni rechaza por autoridad. Se verifica contra documentación técnica primaria (papers, docs oficiales) y se registra el razonamiento en este anexo para trazabilidad ante dirección.

Tabla resumen de retroalimentación

Sugerencia del consultor	Veredicto	Razón principal
Húngaro $O(N \log N)$ en hilo único	Rechazada	Complejidad real $O(N^3)$; SPOF
Mahout + Hadoop para Feature Store	Rechazada	Batch, no apto para inferencia <100 ms
gensim 3.4.0 fijo	Rechazada	Versión 2018 con deuda técnica
Haversine $\times 1.35$, eliminar PostGIS	Rechazada	Error real 15–35 %, no 0.5 %

Equipo: Selene (arquitecta líder) · Aishlinn (datos) · Ana (infraestructura)

Anexo — Evidencia, reunión del equipo

